

# Football Analytics API – Technical Report

---

**Module:** COMP3011 Web Services and Web Data

**Assignment:** Coursework 1 – Individual Web Services API Development Project

**Project:** Football Analytics API (REST API and frontend for football analytics and favourite player lists)

## Links (for submission):

- **GitHub repository:** <https://github.com/Zakari21s/football-analytics-api.git>
  - **API documentation:** [API Documentation \(PDF\)](#) in this repository. Interactive docs at `/docs` (Swagger UI) and `/redoc` when the server is running (or at `base-url/docs` if deployed).
  - **Presentation slides:** *[Insert link to your slides, e.g. Google Drive or OneDrive]*
- 

## 1. Technology stack and justification

| Layer      | Choice                     | Justification                                                                                                                                |
|------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Backend    | Python 3.10+, FastAPI      | Async support, automatic OpenAPI/Swagger, Pydantic validation, clear routing and dependency injection.                                       |
| ORM        | SQLAlchemy 2.x             | Mature ORM with declarative models, session management, and FK support; fits the ERD and CSV-derived schema.                                 |
| Validation | Pydantic                   | Request/response schemas, clear validation errors (422), type-safe config via pydantic-settings.                                             |
| Database   | SQLite                     | Single-file, no separate server; suitable for coursework and local deployment; can be swapped for PostgreSQL via <code>DATABASE_URL</code> . |
| Testing    | pytest, FastAPI TestClient | In-process API tests, fixtures for client and auth headers; no need for httpx async for basic status-code and JSON tests.                    |

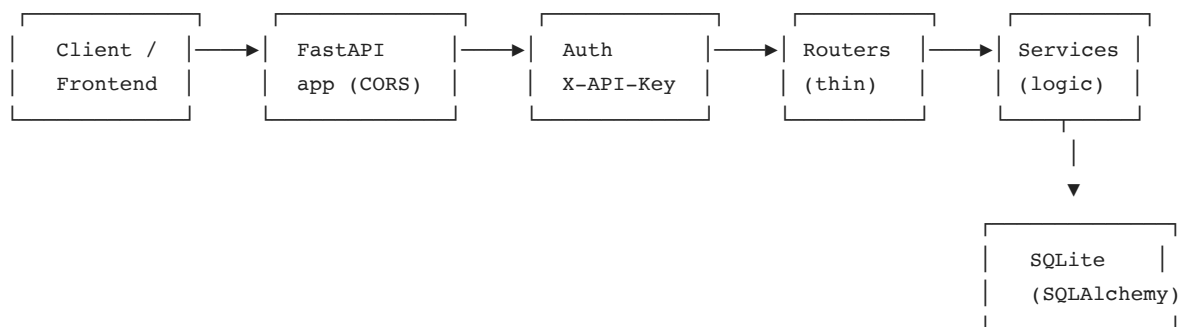
| Layer    | Choice                | Justification                                                                                                                |
|----------|-----------------------|------------------------------------------------------------------------------------------------------------------------------|
| Frontend | Vanilla HTML, CSS, JS | No framework; fetch API with X-API-Key; minimal scope as per brief (players list, favourite lists CRUD, add/remove players). |

Environment configuration is via `.env` and `pydantic-settings`; no secrets in code. The API key is validated in a single dependency and applied to all `/api/v1/*` routes.

---

## 2. Architecture

Request flow is layered: **Client → FastAPI app → Auth (X-API-Key) → Routers → Services → Database**. Routers are thin (parse request, call service, map exceptions to HTTP); business logic lives in services and receives the DB session via dependency injection.



### Key components:

- **app/main.py:** App factory, CORS, mount of API v1 router and frontend at `/app`; `/health` and `/docs` remain on the main app.
- **app/auth:** Single dependency that reads `X-API-Key`, compares to config, returns 401 with a consistent JSON body if missing or invalid.
- **app/routers:** One router per resource (players, teams, favourite\_lists, analytics); all under `/api/v1` with the auth dependency.
- **app/services:** Reusable functions (e.g. `get_players`, `get_top_scorers`) that take a DB session; no HTTP concerns.
- **app/models / app/schemas:** SQLAlchemy models for dataset and application tables; Pydantic schemas for request/response and pagination.

The dataset (players, teams, performances, transfers, market values) is read-only from the API; the only full CRUD resource is **FavouriteList**, with **FavouriteListPlayer** used for many-to-many membership (add/remove/view list players).

---

### 3. Design choices

**REST and URLs:** All API endpoints are under `/api/v1` and use consistent trailing slashes (e.g. `/api/v1/players/` , `/api/v1/favourite-lists/` ) to avoid redirects. Resource-based paths: e.g. `GET /api/v1/players/` , `GET /api/v1/players/{id}/details/` , `POST /api/v1/favourite-lists/` , `GET /api/v1/favourite-lists/{id}/players/` .

**Status codes:** 200 (OK), 201 (Created for POST), 204 (No Content for DELETE), 400/422 (validation), 401 (missing/invalid API key), 404 (resource not found), 409 (e.g. player already in list). Error responses use a consistent JSON shape (e.g. `{"detail": {"message": "...", "code": "..."} }` ).

**CRUD resource:** The main database-backed CRUD resource is **FavouriteList** (create, read list, read one, update name, delete). **FavouriteListPlayer** supports only add player, remove player, and list players (with optional sort); it is not a separate full CRUD resource.

**Pagination:** List endpoints that return many items (e.g. players, performances, transfers) use `page` , `limit` , and respond with `{ "data": [...], "page", "total_pages", "total_count" } .`

**Authentication:** Every `/api/v1/*` request must include a valid `X-API-Key` header; the dependency runs before route handlers. `/health` , `/docs` , and `/openapi.json` are unauthenticated.

---

### 4. Implementation highlights

**Sorting (players and list players):** The API supports `sort_by` (e.g. `name` , `age` , `market_value` , `minutes_played` ) and `order` (`asc`/`desc`). Age is derived from `date_of_birth` ; market value from the latest row per player in `player_market_value` (subquery on max date); minutes from `SUM(minutes_played) OVER player_performances` . The same logic is used for listing players inside a favourite list.

**Analytics:** Three endpoints demonstrate use of the football dataset: (1) **top-scorers** – `SUM(goals)` from performances, optional filter by season/competition\_id; (2) **top-market-values** – latest value per player, ordered by value desc; (3) **most-minutes-**

**played** – `SUM(minutes_played)` per player, optional season filter. All return a list of player identifiers and the computed metric.

### Advanced analytics:

- **Player details** – `GET /api/v1/players/{id}/details` returns current market value, full market value history, and a career summary. Current market value is the latest value per player from `player_market_value` (row with max date per player). Market value history is all rows for that player ordered by date. Career summary includes seasons played (distinct seasons from `player_performances`) and previous clubs (distinct team names from performances joined with teams, or from transfer history). One combined response keeps the frontend modal simple.
- **New analytics endpoints – top-assists:** sum of `assists` per player from `player_performances`, with optional season and competition filters, returned in descending order. **Youngest-stars:** players under a configurable age limit, with aggregated `total_minutes` and `total_goals` from performances, sorted by minutes then goals. Both use the same `PlayerPerformance` (and player) data as the existing analytics endpoints.

### Alternatives considered:

- **REST vs GraphQL:** REST was chosen for simplicity and strong tooling (OpenAPI, Swagger/ReDoc). GraphQL would suit flexible client queries but was not required for this scope.
- **SQLite vs PostgreSQL:** SQLite for single-file, no separate server, and easy local use; for production a server DB such as PostgreSQL with migrations (e.g. Alembic) would be preferred.
- **One details endpoint vs several:** A single combined details endpoint was chosen for the player modal. Splitting into e.g. `/market-value-history` and `/career` would allow finer-grained caching but was not needed for the current frontend.

**Dataset pipeline:** Data is filtered to top-five leagues (GB1, ES1, IT1, L1, FR1) in a dedicated script that builds allowed club/player IDs and writes filtered CSVs to `web/filtered/`. A separate import script loads these in FK order (teams → players → performances, transfers, market values) with batched commits and streaming reads to avoid loading entire files into memory.

**Frontend scope:** The single-page frontend includes a players table (sort, order, pagination, search, league/season filters), favourite lists (create/list/delete lists, card-based list selection, add/remove players by search, cards/table view), an analytics tab

(top scorers, assists, market values, minutes, youngest stars with filters), and a player-details modal (current snapshot, career summary with previous clubs and logos, market value history chart). API key is set via the header bar; all API calls use the `X-API-Key` header. CORS is configured for local and Codespaces origins so the frontend works when served from different origins.

---

## 5. Testing approach

Tests use `pytest` and FastAPI's `TestClient` (no separate server). **confest.py** provides a shared client and `auth_headers` (valid API key). The same SQLite database used for local development is used for tests (no in-memory override in the current setup).

**Coverage:** (1) **Auth** – request without key → 401; wrong key → 401; valid key → 200 on a protected route. (2) **Favourite lists CRUD** – full cycle (create, get, patch, delete, then 404); 404 for get/patch/delete on missing id. (3) **Favourite list players** – add player, list players, add same again → 409, remove player, list empty; 404 when list or player does not exist. (4) **Validation** – POST favourite list with empty or missing name → 422; add player with non-existent `player_id` → 404. (5) **Players** – list returns 200 and structure `data, page, total_pages, total_count; sort_by=name`; get by id 200/404. (6) **Analytics** – GET top-scorers (and other analytics endpoints) returns 200 and list of items with expected fields.

Tests are run with `pytest` or `pytest tests/ -v`; see README.

### 5.1 Challenges and lessons learned

- **API contract and clients:** Using consistent trailing-slash URLs and CORS for both local and Codespaces origins avoided 307 redirects and preflight failures; documenting base URLs and headers in the README and API docs reduced integration issues.
- **Data shape and performance:** Aggregations (market value, minutes, goals) are computed at request time; for larger datasets, materialized views or cached aggregates would be worth considering. The single player-details endpoint simplified the frontend at the cost of a larger response; the trade-off was acceptable for this scope.
- **Testing:** Relying on FastAPI's `TestClient` and a shared SQLite DB kept tests simple and fast. Adding tests for new endpoints (details, analytics) alongside existing CRUD tests helped catch regressions early.

- **GenAI-assisted development:** Using AI for design and implementation sped up development; verifying every suggestion (security, status codes, validation) and running tests after changes ensured correctness and understanding.
- 

## 6. Deployment

Deployment has **not** been performed for this submission. The project runs locally with SQLite. The README describes how to run the server ( `uvicorn app.main:app` ), create the DB, load data, and run tests. When deployment is performed, replace this paragraph with one or two sentences: where the app is deployed (e.g. PythonAnywhere or Render), that `DATABASE_URL` and `API_KEY` are set, and that `create_db` and `load_data` were run once. Add the live API URL (and frontend URL if applicable).

---

## 7. Use of Generative AI

Generative AI (Cursor with Claude/Codex) was used for architecture and API design, code generation (backend and frontend), testing, and documentation. Every suggestion was reviewed, adapted where necessary, and verified (e.g. by tests or manual checks). The use was methodical and aligned with the coursework rules: tools and purposes are declared, sample conversation excerpts are summarised, and reflection on use is provided. **Appendix A** below points to the full declaration and supplementary material; the canonical GenAI declaration is in `docs/GENAI_DECLARATION.md` in the repository. Exported conversation logs are provided as supplementary material as required by the brief.

---

## 8. Limitations and future work

- **Single API key:** Authentication is a single shared key; there is no per-user identity or rate limiting.
- **SQLite:** Suitable for coursework and small scale; for production, a server DB (e.g. PostgreSQL) and proper migrations (e.g. Alembic) would be preferable.
- **Frontend:** The frontend includes players table, favourite lists (card-based UI), analytics, and player-details modal; further improvements could include richer error feedback, loading states, and broader responsive layout.
- **Data:** Dataset is static after import; no live sync with an external source. Analytics are computed at request time (subqueries/aggregations); materialized views or

cached aggregates could improve performance for heavy use.

---

## Appendix A: GenAI declaration and supplementary material

The full **Generative AI declaration** is in `docs/GENAI_DECLARATION.md` in this repository. It includes:

1. **Tools used and purpose** – Cursor (AI-assisted editor) and Claude/Codex via Cursor, with declared purposes (architecture, code generation, CORS/frontend, tests, documentation).
2. **Sample conversation excerpts** – Summaries of representative exchanges (API key and router design, player details endpoint, analytics endpoints, test cases, CORS and frontend behaviour).
3. **Reflection and analysis** – Methodological use of GenAI, verification and adaptation of suggestions, and use for creative or solution-level tasks (e.g. endpoint design, frontend layout).
4. **Supplementary material** – Exported conversation logs are provided as separate files (or in the submission package) as required by the coursework brief.

For the single-PDF submission, either (a) append the contents of `GENAI_DECLARATION.md` to this report after this appendix, or (b) ensure the submission package includes both this technical report PDF and the GenAI declaration (and conversation log examples) as specified in the brief.

---

*Report length: main body within the recommended page limit; appendix as above.  
Export this document (and, if required, the GenAI declaration) to a single PDF for Minerva submission.*